

C Programming (W2)



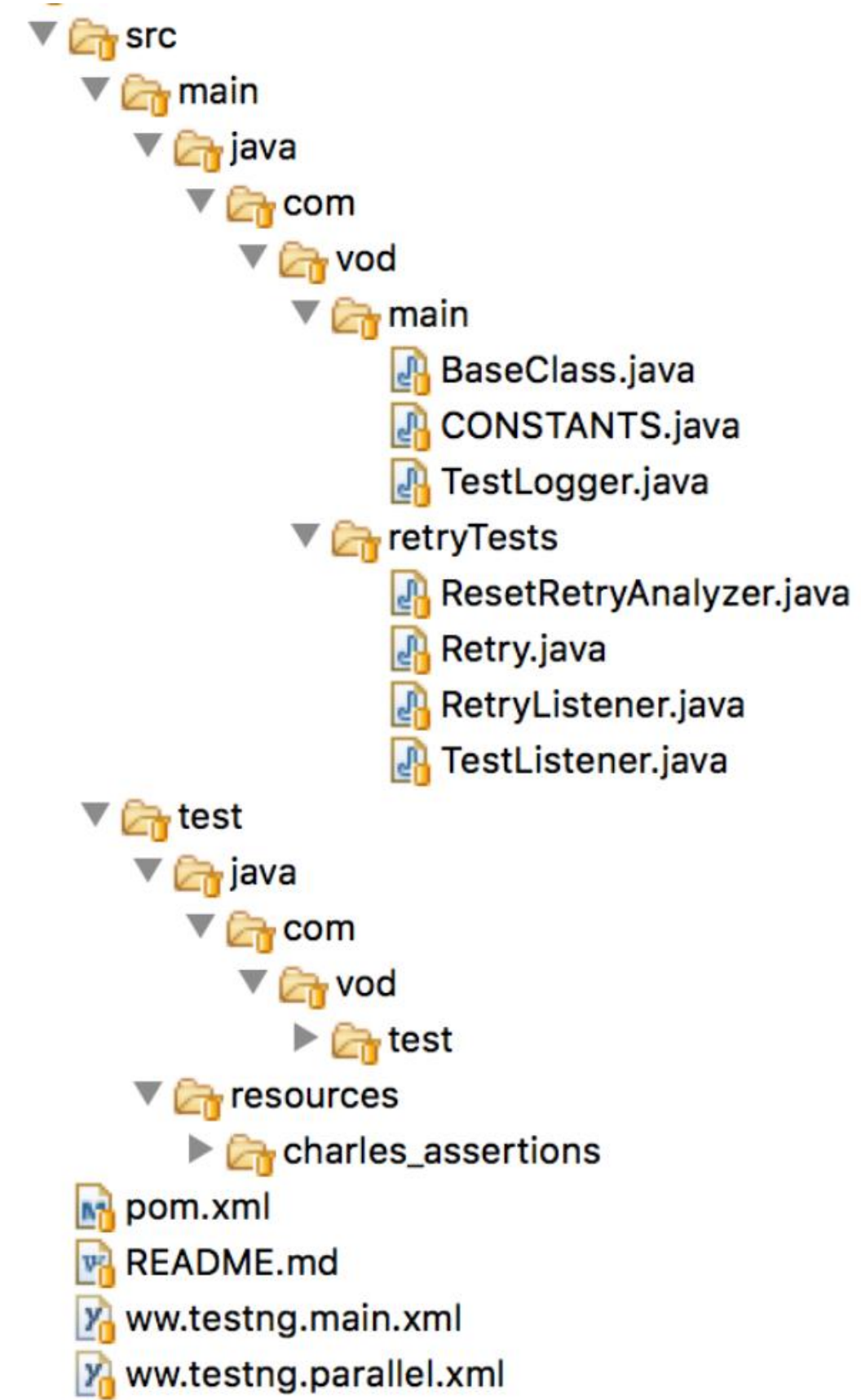
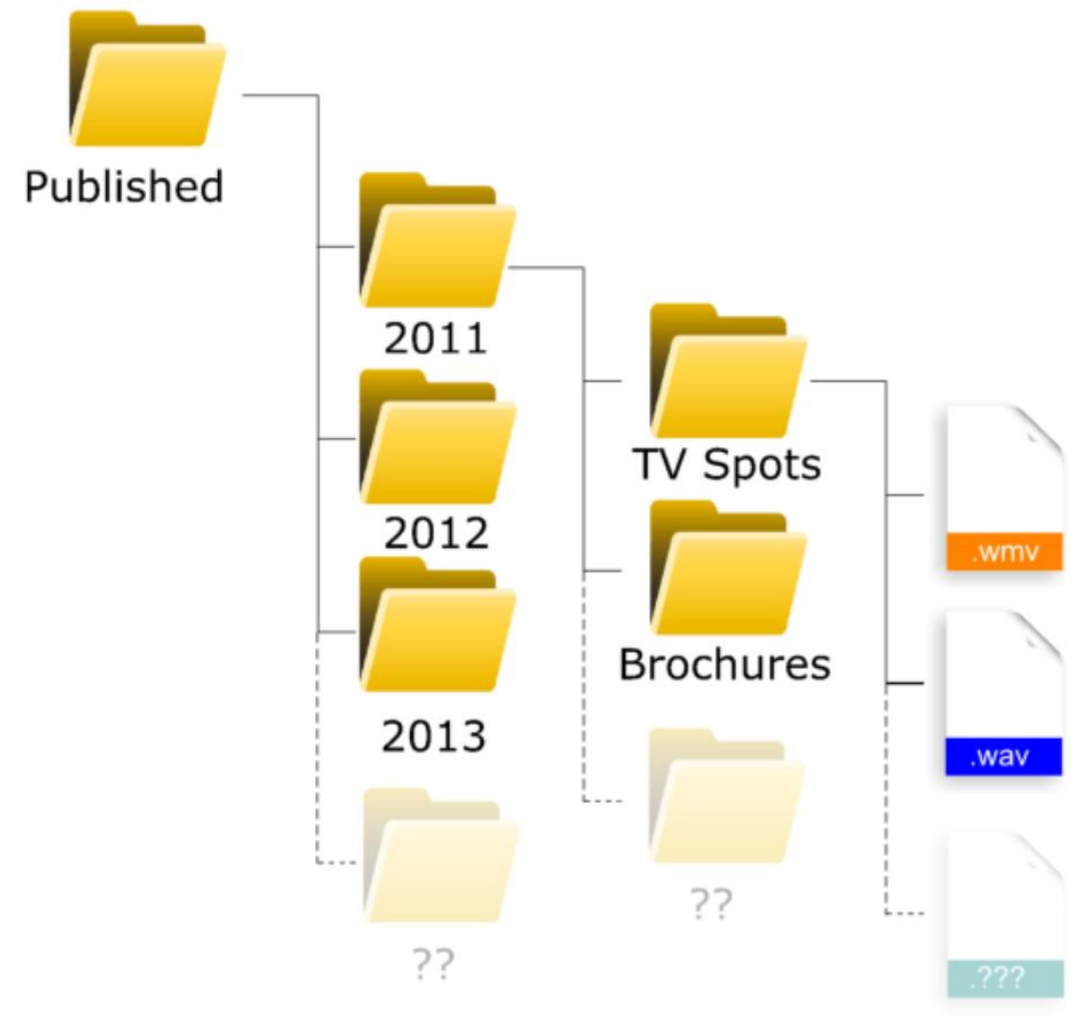
Welcome!!

Please check attendance individually.
(Mobile App)

Things to do today

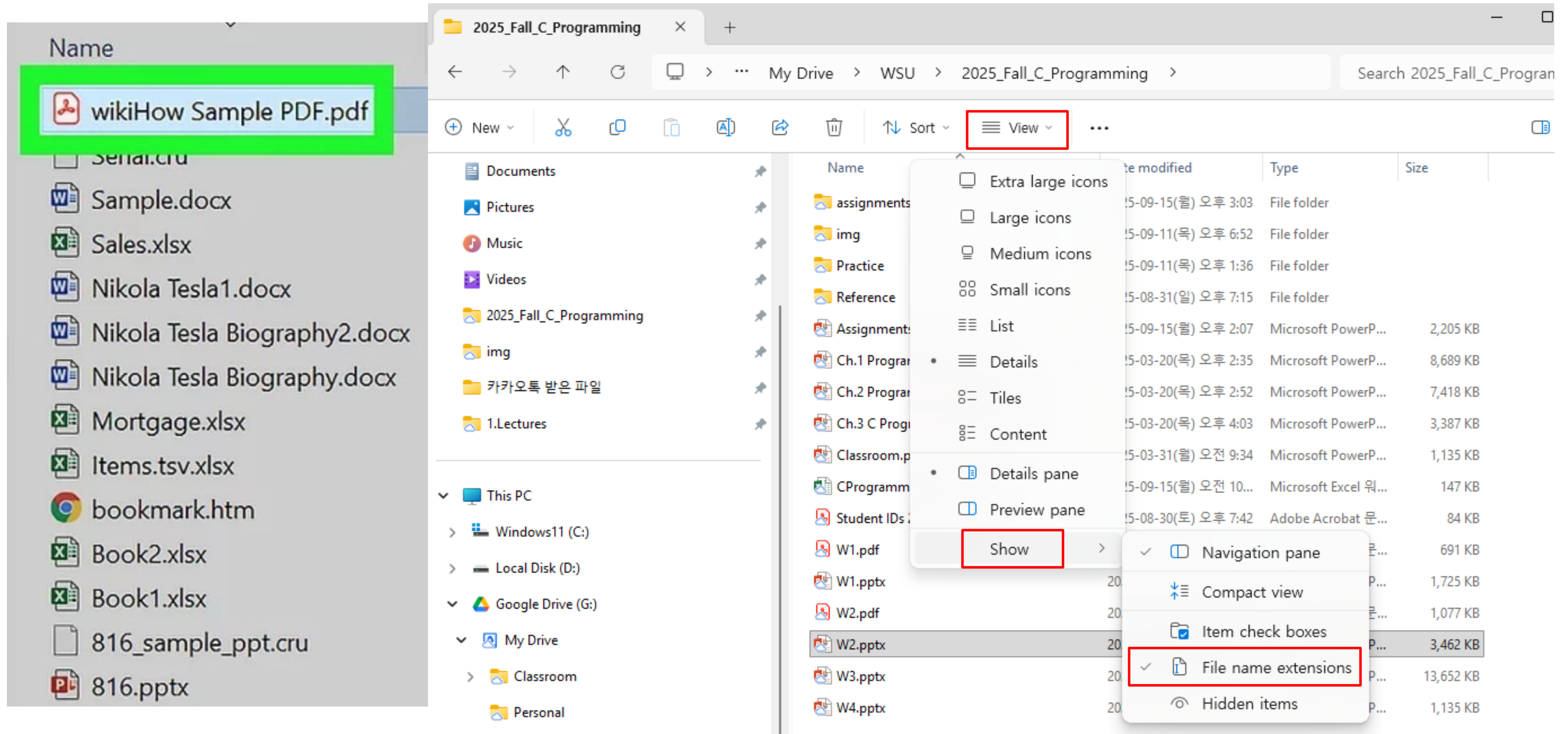
- 01** | Folder & File / File extension
- 02** | Shell
- 03** | First Program & C compile process
- 04** | Standard Input/Output : printf & scanf

Folder & File



File extension

- *running a file using the metaphor of calling a person's name*



Shell

A **shell** is like a **translator between you and the operating system (OS)**.

It takes the commands you type, sends them to the OS, and then shows you the results.

In a PC, a **shell** is a user interface that allows you to interact with the operating system. It can be **command-line-based (CLI)** or **graphical (GUI)**:

1. Command-Line Shell (CLI) – This is a text-based interface where users enter commands to perform tasks. Examples:

- **Bash** (Linux, macOS)
- **PowerShell** (Windows)
- **Command Prompt (cmd.exe)** (Windows)
- **Zsh, Fish** (Unix-like systems)

2. Graphical Shell (GUI) – This provides a visual interface with icons, windows, and menus. Examples:

- **Windows Explorer** (Windows shell)
- **GNOME/KDE/Xfce** (Linux desktop environments)
- **macOS Finder** (Mac shell)

Terminology

- **Terminal**(room) is the program that opens a window to interact with the system.
 - A **terminal** is the window/app where you type commands and see the text output.
 - It's the interface for interacting with a shell.
 - **Console**: historically the physical I/O; today often used loosely like “terminal.”
- **Shell**(teacher) is the command interpreter that runs inside the terminal and translates user commands into actions performed by the operating system.

“Open windows terminal using Command Prompt shell”

Common Windows Shell Commands

Command	Description	Example
dir	List files and directories in current folder	dir C:\Users
cd [path]	Change directory (.. = up one level)	cd C:\Windows
mkdir [name]	Create a new folder	mkdir Projects
rmdir [name]	Remove a folder (must be empty)	rmdir Temp
del [file]	Delete a file	del notes.txt
copy [src] [dest]	Copy a file	copy a.txt D:\Backup\
move [src] [dest]	Move or rename a file	move old.txt new.txt

Useful tools

Command	Description	
doskey /history	A tool used in Command Prompt (cmd) to create command macros or to view the history of previously entered commands	
Where [file]	Displays the location of the executable file for a given command or program	

System & Info

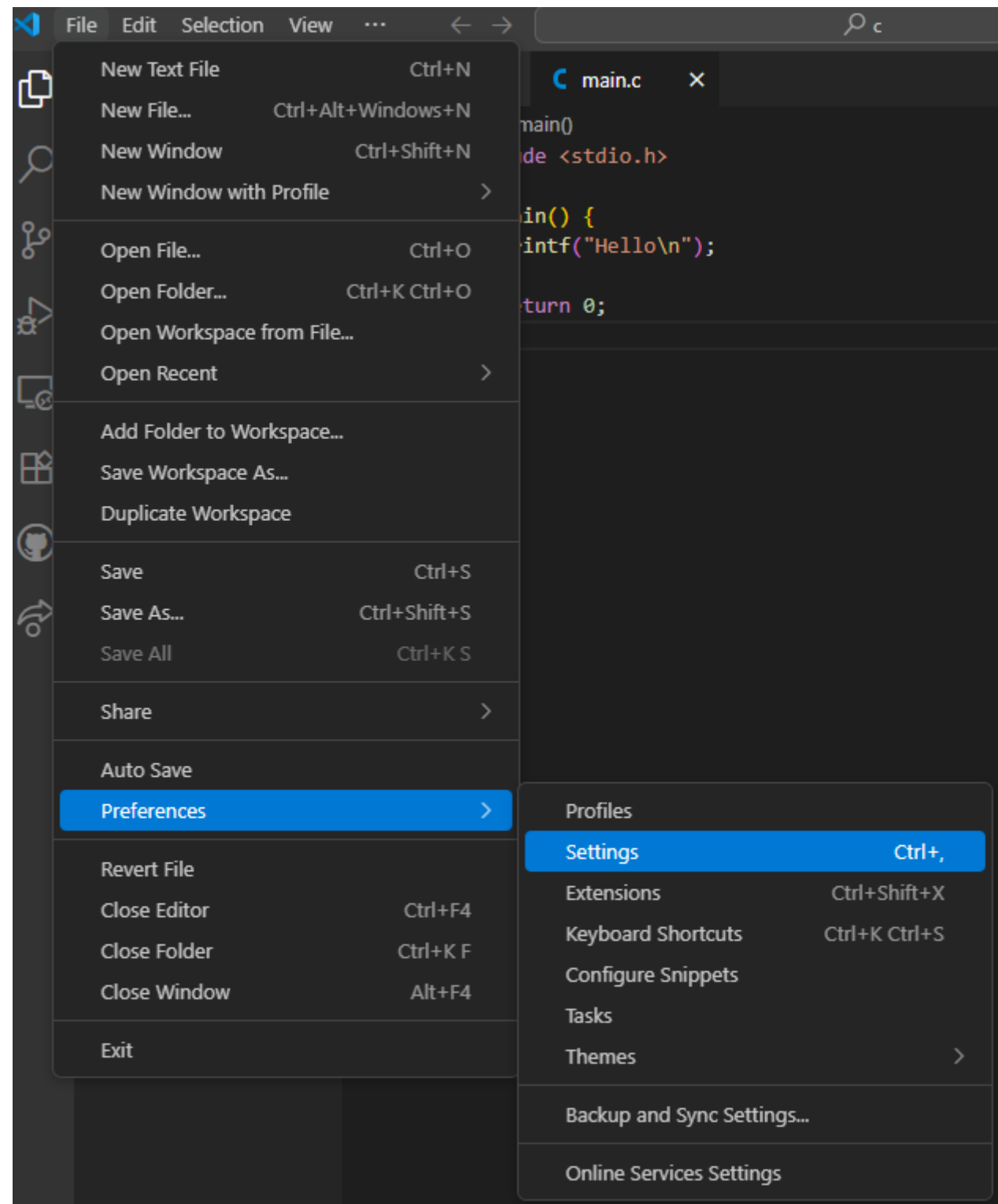
Command	Description	Example
cls	Clear screen	cls
echo [text]	Print text	echo Hello World
systeminfo	Show system info	systeminfo
hostname	Show computer name	hostname
ipconfig	Show IP/network info	ipconfig /all
ping [address]	Test network connection	ping google.com
tasklist	Show running processes	tasklist
taskkill /PID [id] /F	Kill process by ID	taskkill /PID 1234 /F

Networking

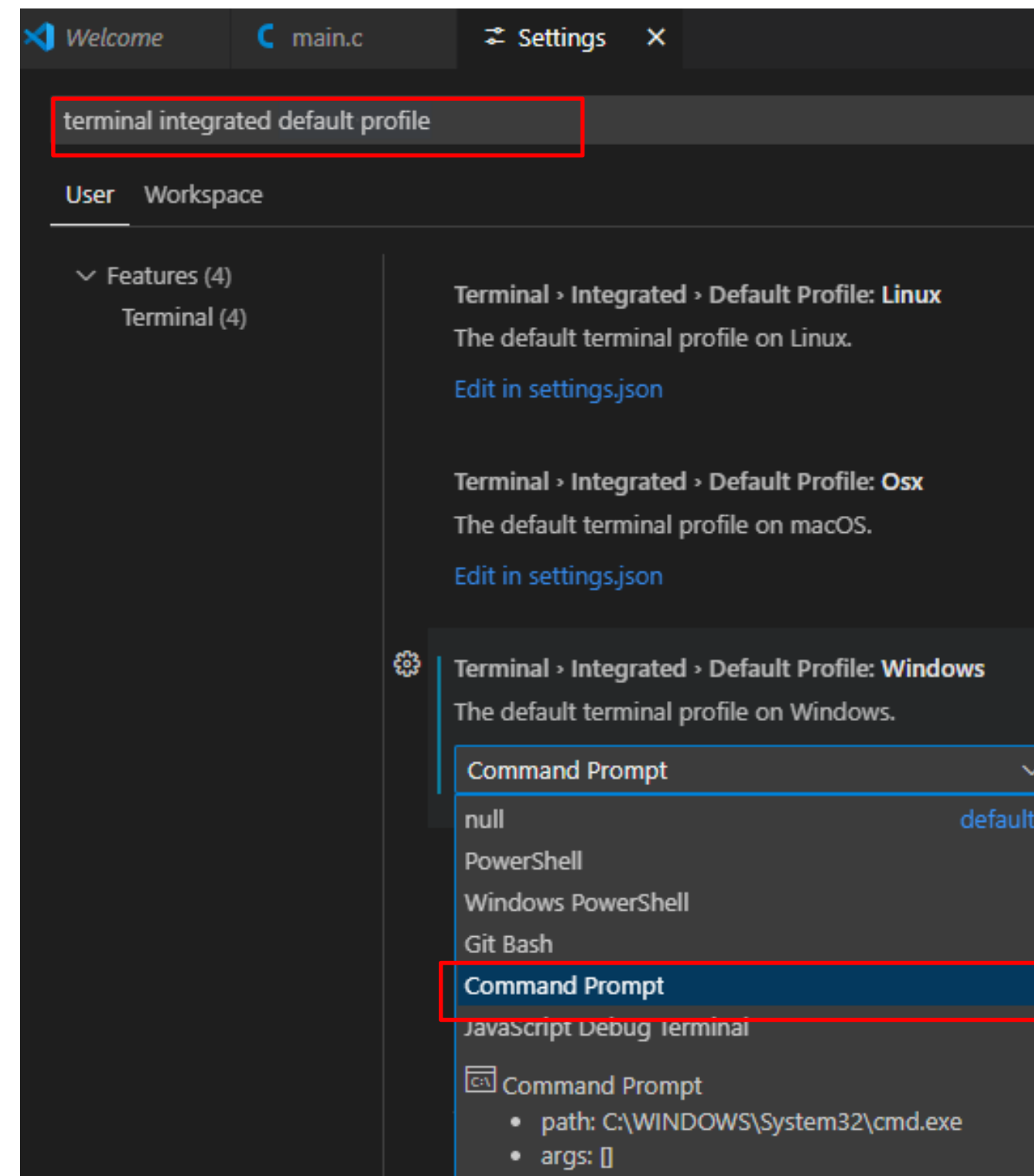
Command	Description	Example
netstat	Show active connections	netstat -an
tracert [addr]	Trace route	tracert google.com
nslookup [domain]	DNS lookup	nslookup openai.com

How to change default shell in VSC

. Setting > Terminal > default profile: Windows > Command Prompt



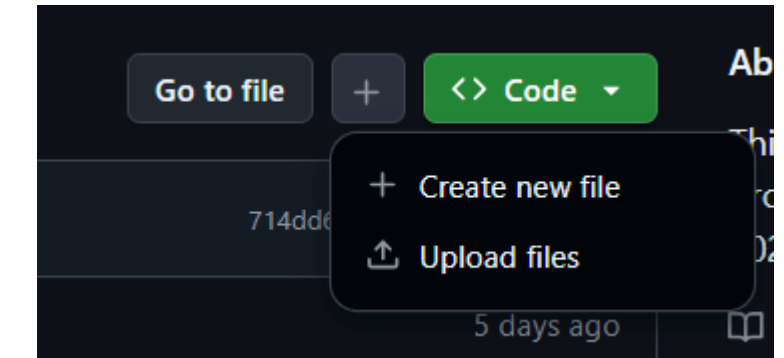
TYPE: terminal integrated default profile



How to Manage Files on GitHub

[Method]

1. Go to the **personal repository** that has been created for you.
2. On the repository page,
 - To create a new file: “**Add file** → **Create new file**” → then enter the file name and Contents.
 - To upload files: Click “**Add file** → **Upload files**”, then and drag-and-drop your files.
 - To edit existing files: Open the file and click the **pencil icon (Edit this file)**.
3. To save your changes, enter a **commit message** and click “**Commit changes**”.
4. All changes are automatically saved to the repository and and will be visible to the instructor.



Development Environment & setup - Make the first my program

01 | @ VSCode with terminal

02 | @ Codespaces (online)

Development Environment & setup - Codespaces

1. Goto your personal repository of github

The screenshot shows the GitHub interface for a repository named 'practice' owned by 'prof-kweon'. The repository is public and has 2 stars, 1 watch, and 2 forks. The main branch is selected. The file list shows several files: 'New Mindmap.mm', 'README.md', 'about_mc.c', 'about_me.c', 'about_me2.c', 'about_me3.c', 'main.c', 'main2.c', and 'practice_stdio.c'. The 'main2.c' file was created 5 days ago. The 'About' section on the right indicates no description, website, or topics are provided. The 'Releases' section shows no releases published. The 'Packages' section shows no packages published. The 'Languages' section shows C as 100.0%.

prof-kweon / practice

Type / to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

practice Public

Pin Unwatch 1 Fork 2 Star 2

main Go to file

prof-kweon Create main2.c 9c779d4 · 5 days ago 14 Commits

New Mindmap.mm	Add files via upload	6 months ago
README.md	Initial commit	6 months ago
about_mc.c	Create about_mc.c	6 months ago
about_me.c	Create about_me.c	6 months ago
about_me2.c	Update about_me2.c	6 months ago
about_me3.c	Create about_me3.c	6 months ago
main.c	Create main.c	6 months ago
main2.c	Create main2.c	5 days ago
practice_stdio.c	Update practice_stdio.c	6 months ago

README

About

No description, website, or topics provided.

- Readme
- Activity
- 2 stars
- 1 watching
- 2 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

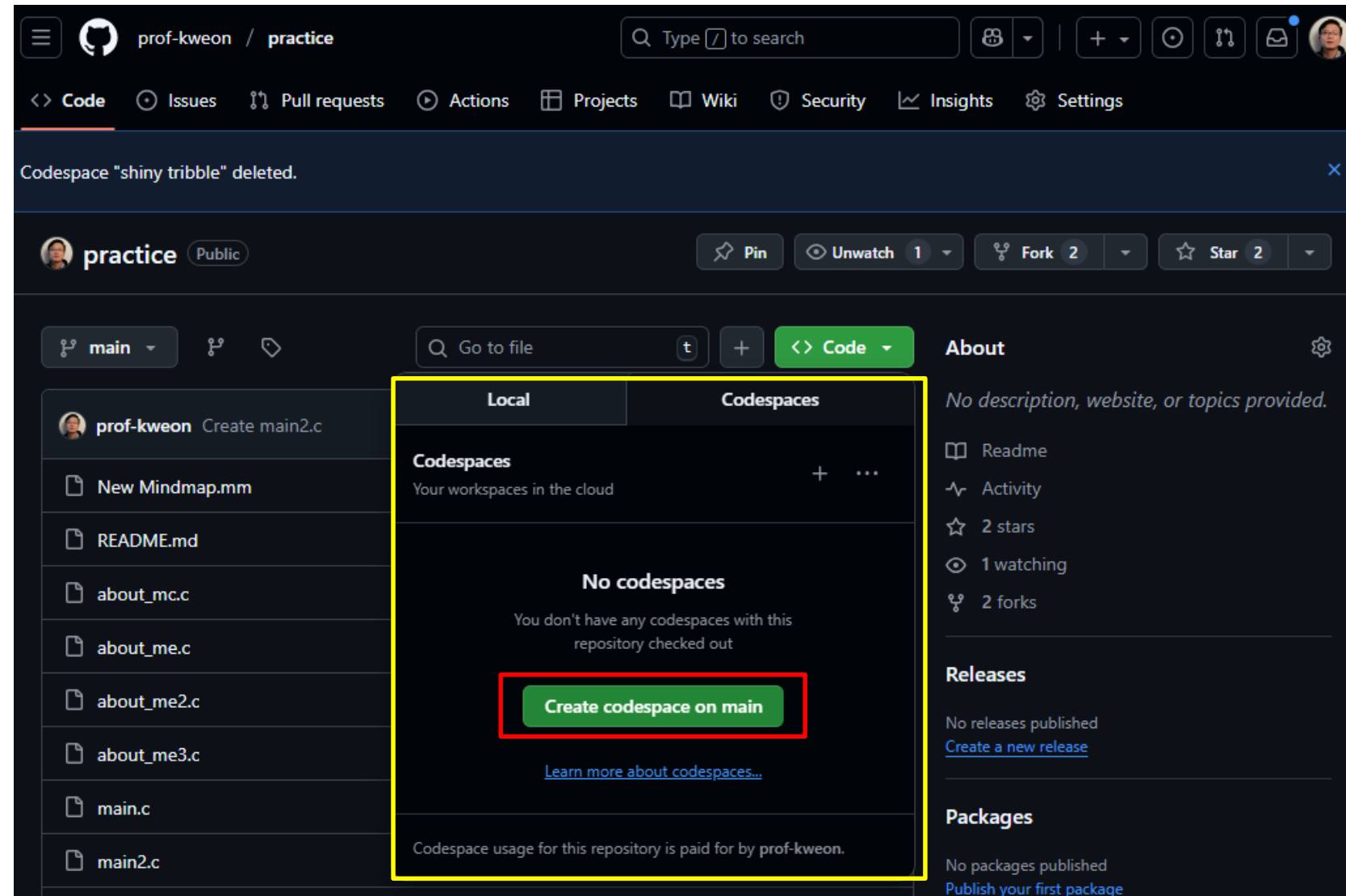
[Publish your first package](#)

Languages

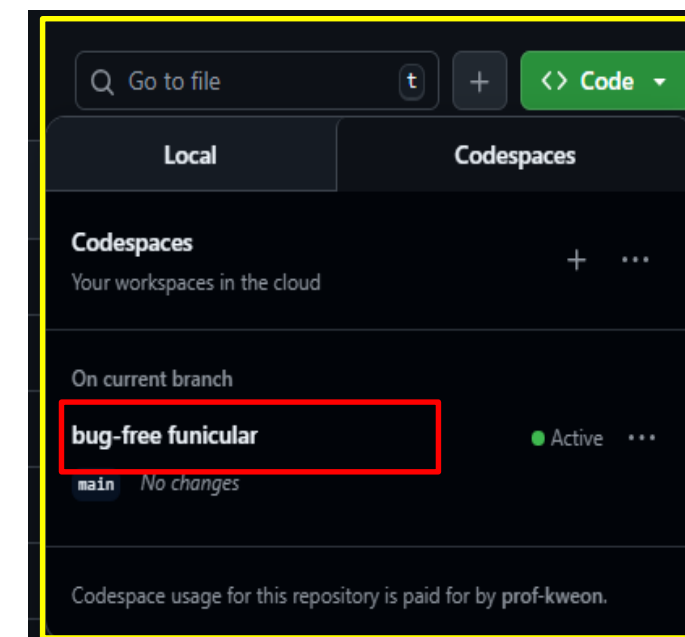
C 100.0%

Development Environment & setup - Codespaces

2. Click “code”, select “Codespace” tab, then “Create codespace on main”

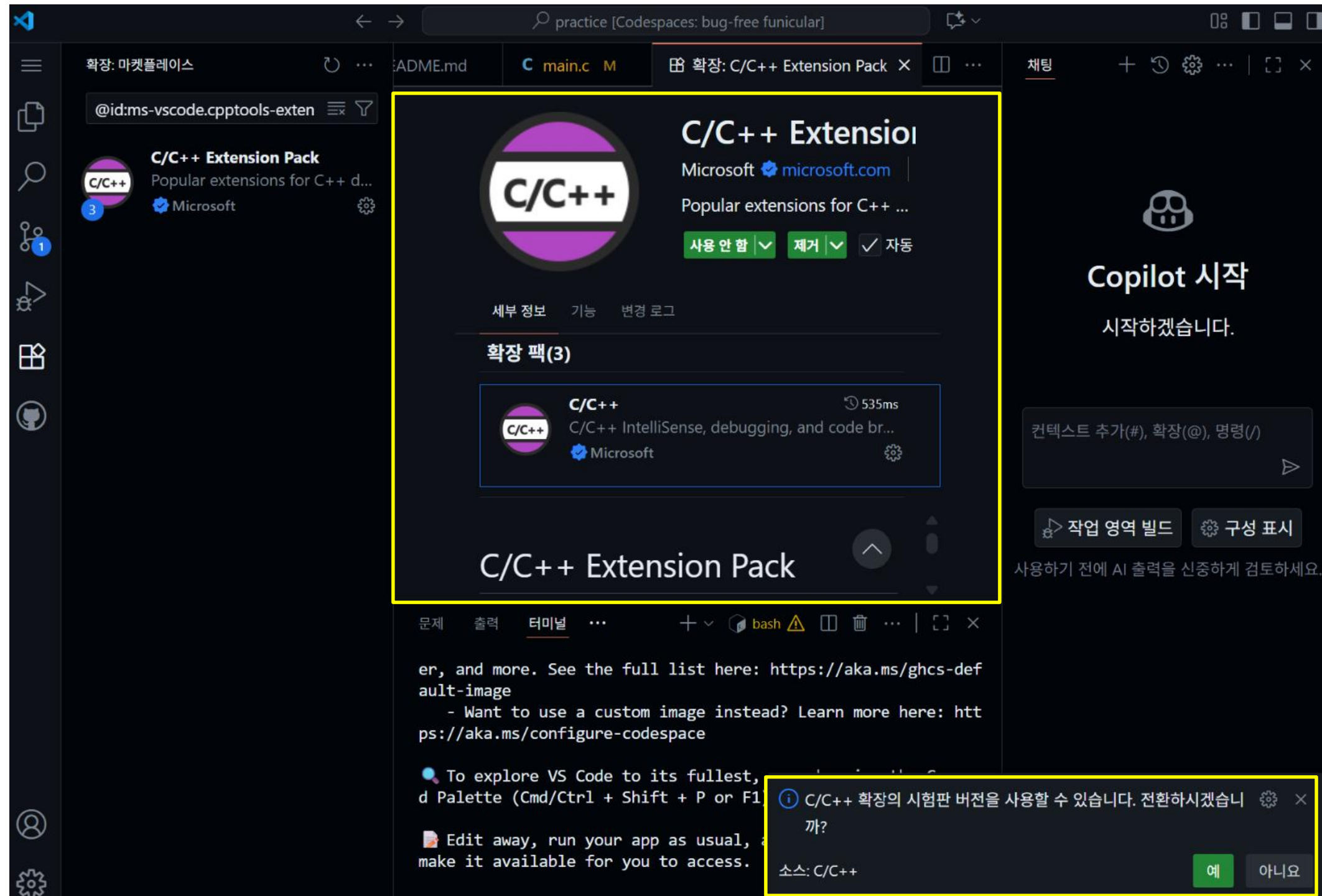


2-1. If you already created the codespace with certain repository, select “On current branch”



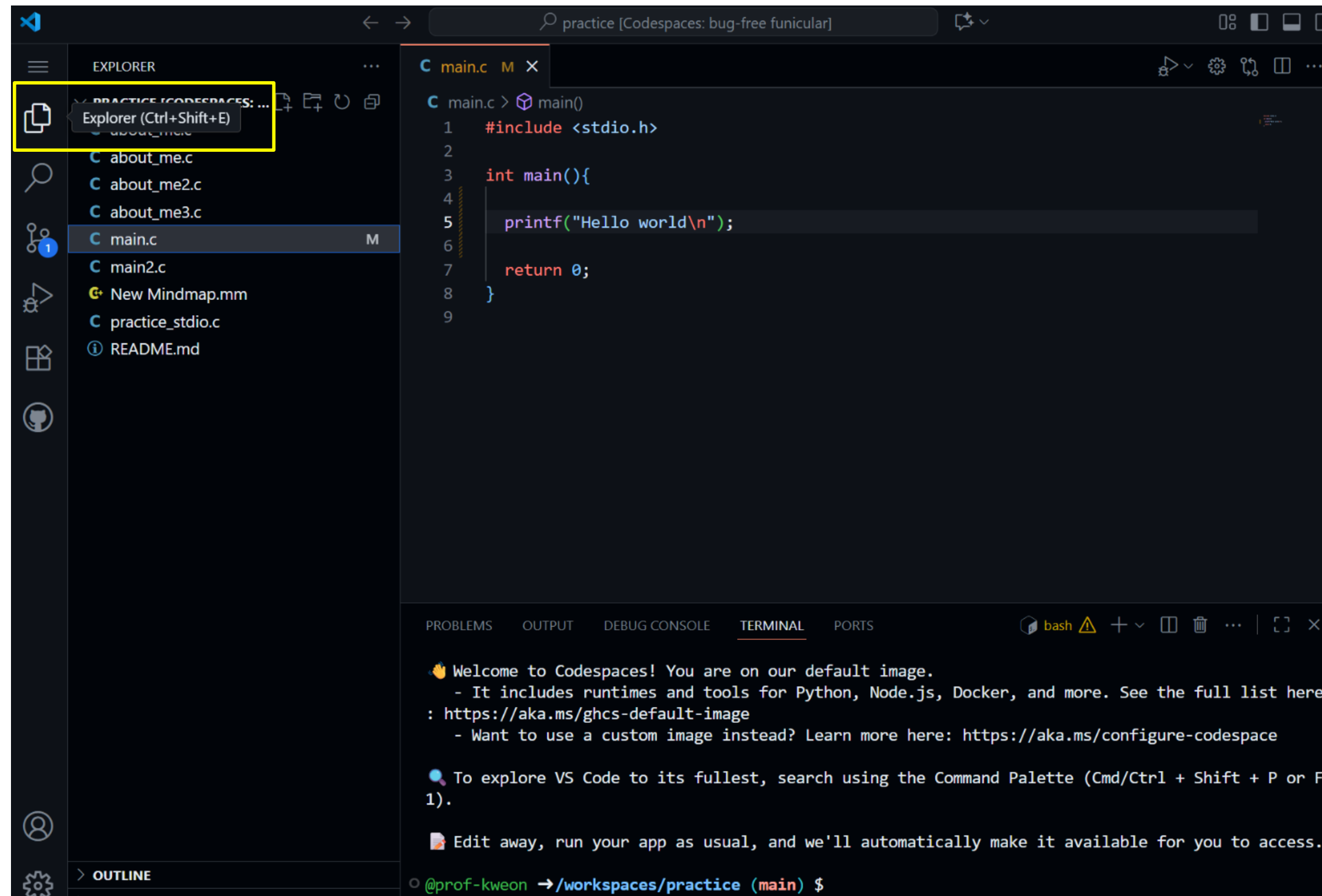
Development Environment & setup - Codespaces

3. When you got the popup messages from codespaces : install or switch C/C++ extension..... > install / accept them



Development Environment & setup - Codespaces

4. You can view the files in the current folder by clicking the **Explorer** button in the upper-left corner.



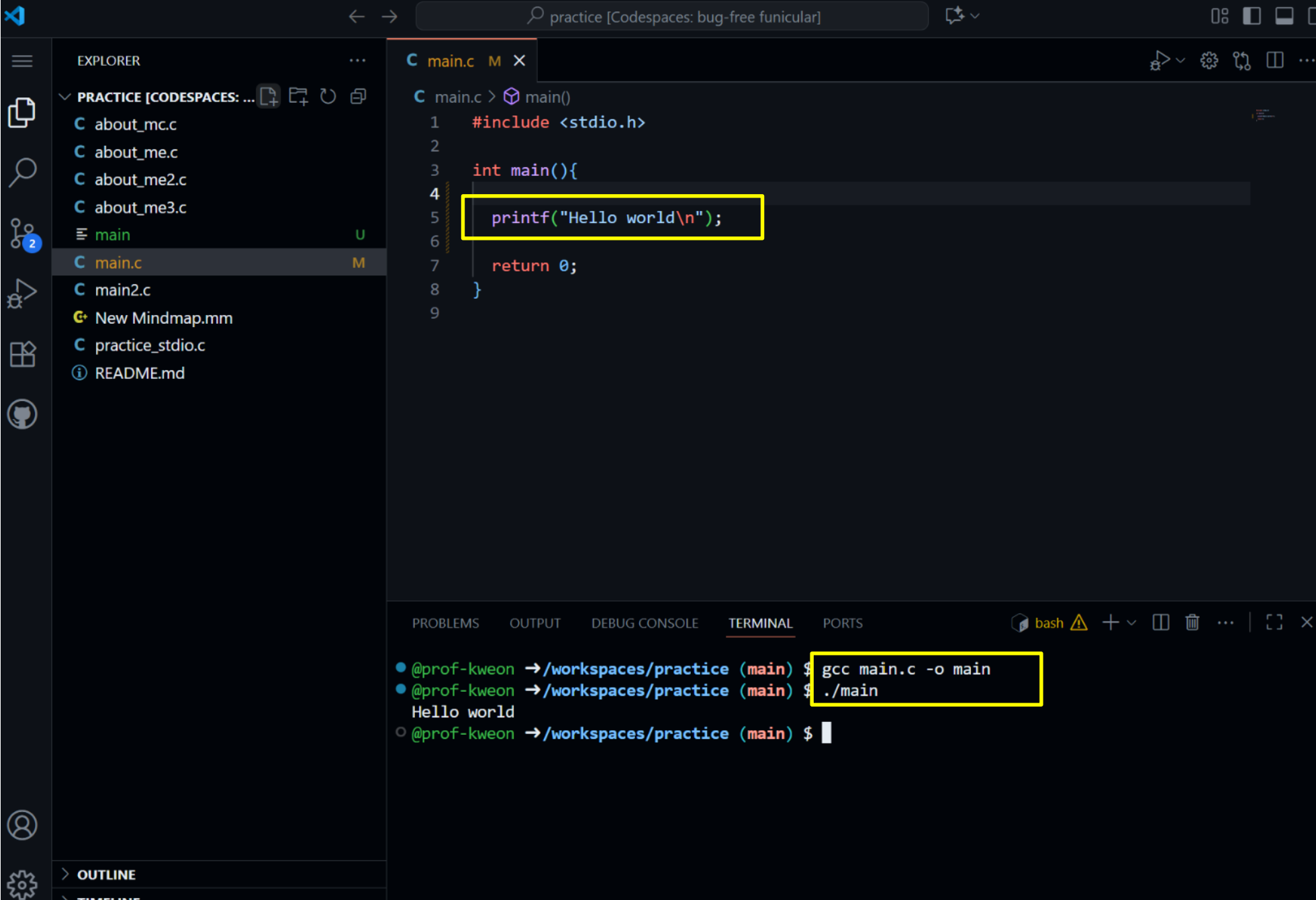
Development Environment & setup – Codespaces (Modify & Verify)

5. Now modify main.c (It might be saved when you compile the file. To make sure, press ctrl+s)

6. Compile & run main.c at terminal which is bash shell

```
gcc main.c -o main
./main
```

7. Verify your source codes



The screenshot shows a VS Code interface for a Codespace named 'practice [Codespaces: bug-free funicular]'. The Explorer panel on the left lists files: about_me.c, about_me2.c, about_me3.c, main, main.c, main2.c, New Mindmap.mm, practice_stdio.c, and README.md. The main.c file is open in the editor, showing the following code:

```
1 #include <stdio.h>
2
3 int main(){
4     printf("Hello world\n");
5
6     return 0;
7 }
8
9
```

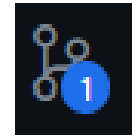
The terminal at the bottom shows the execution of the code:

```
@prof-kweon →/workspaces/practice (main) $ gcc main.c -o main
@prof-kweon →/workspaces/practice (main) $ ./main
Hello world
@prof-kweon →/workspaces/practice (main) $
```

Development Environment & setup – Codespaces (Commit - GUI)

8. Commit your source codes to the personal repo.

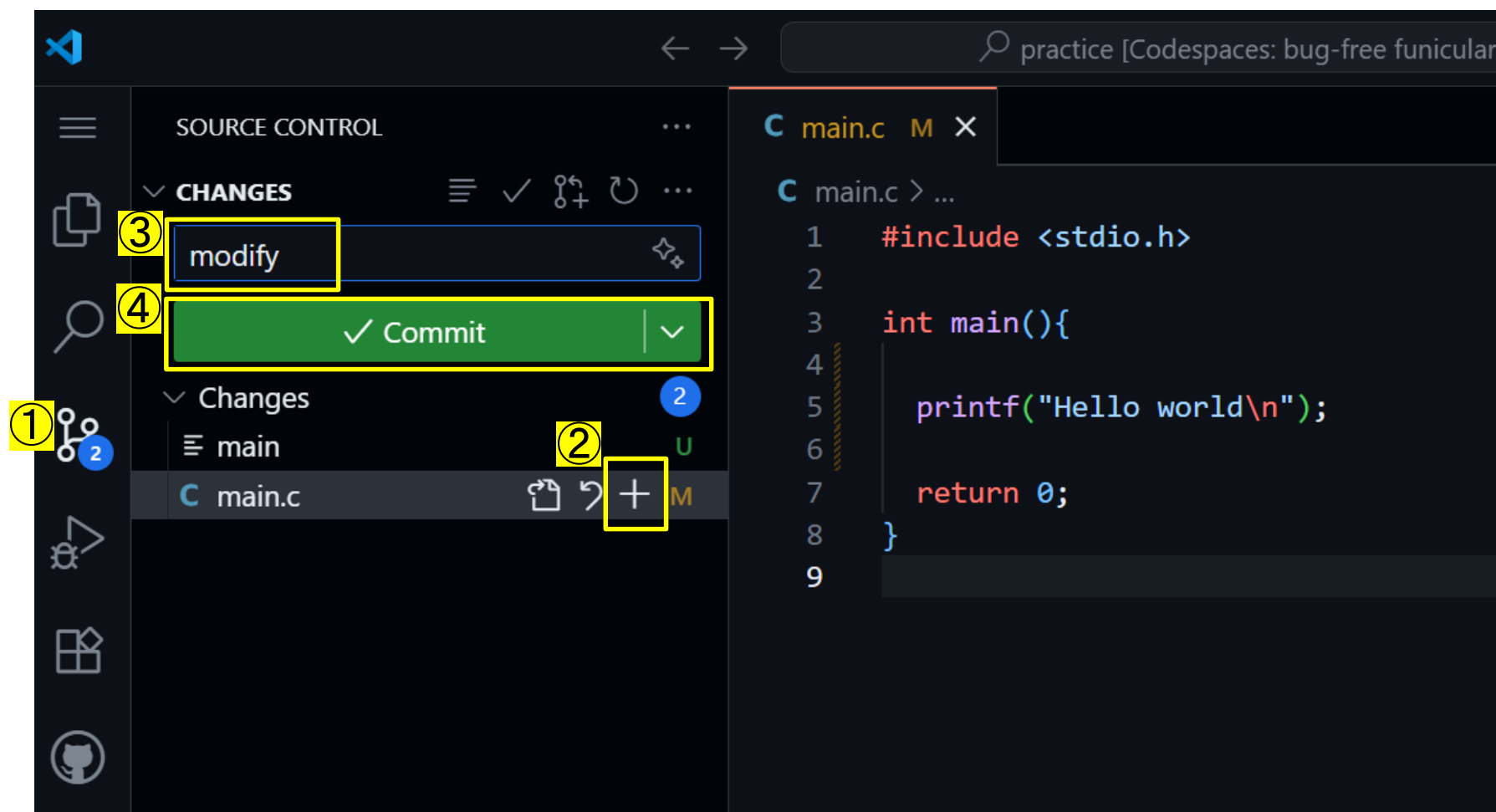
8-1. Press “Source Control” button



8-2. Press “+” button to commit

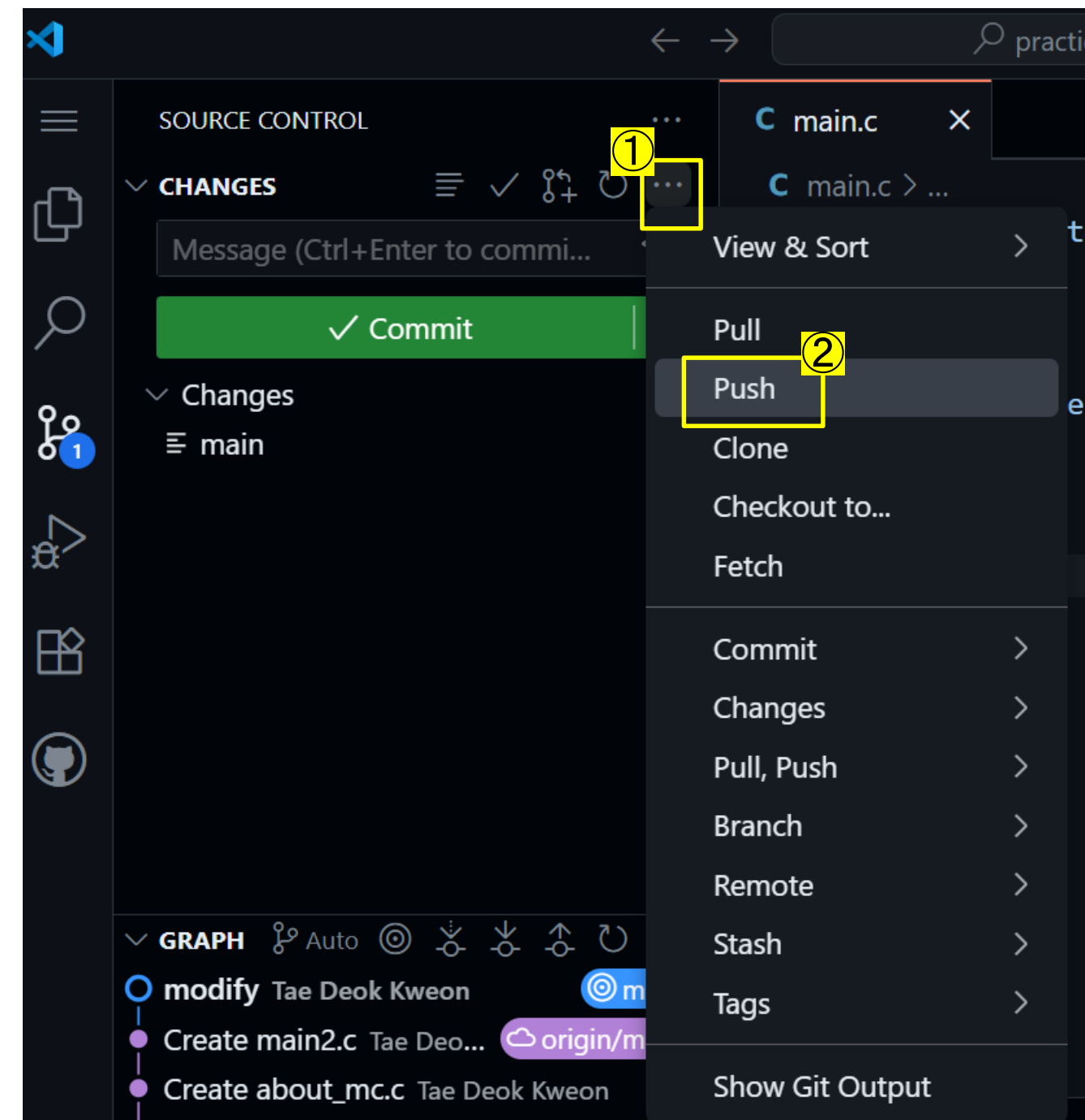
8-3. write any message for committing

8-4. Press “Commit” button



9. Press “...”, then “Push”

10. Close browser tab & reopen your github repo.



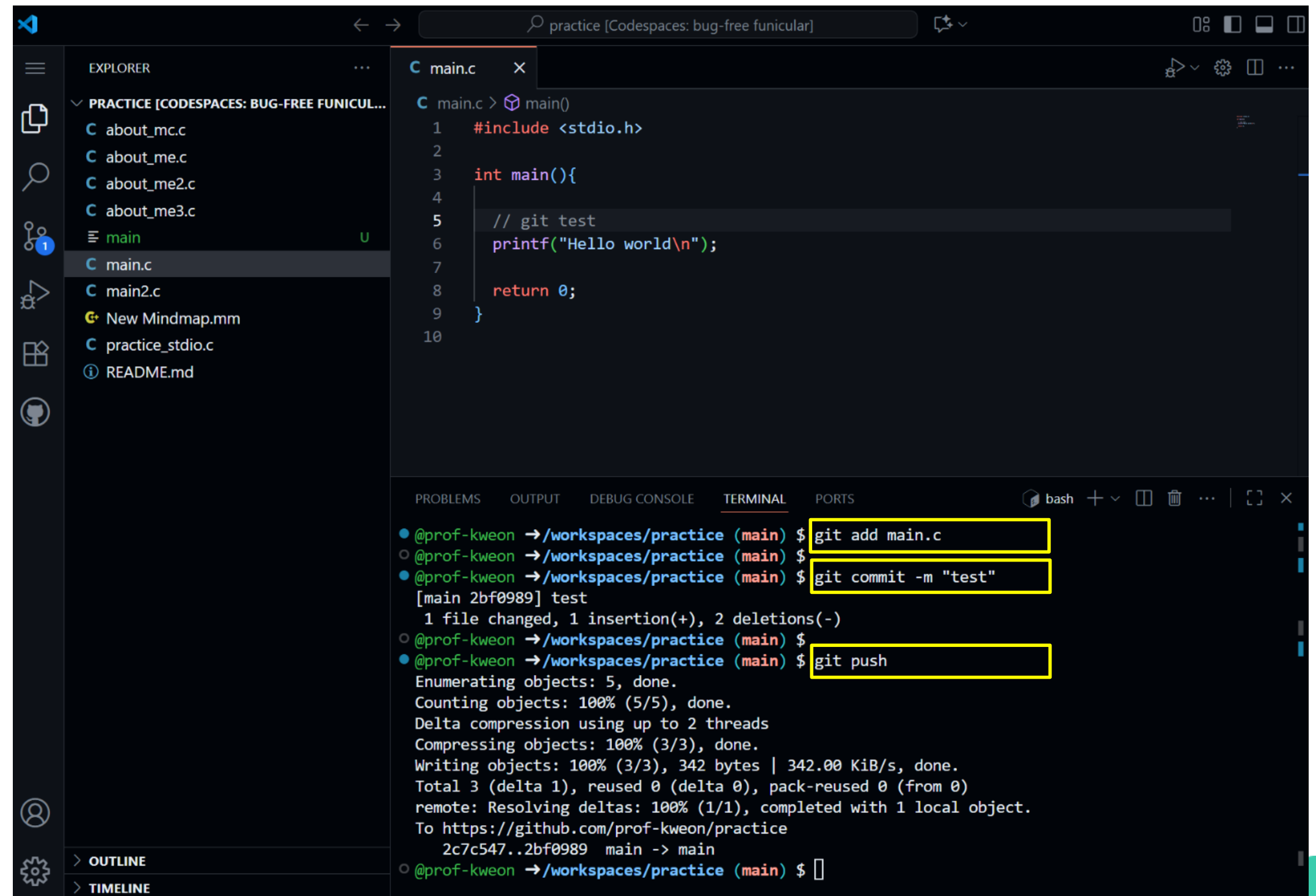
Development Environment & setup – Codespaces (Commit - CLI)

8. After modification & verification, type the following.

```
git add . (git add main.c)
```

```
git commit -m "msg"
```

```
git push
```



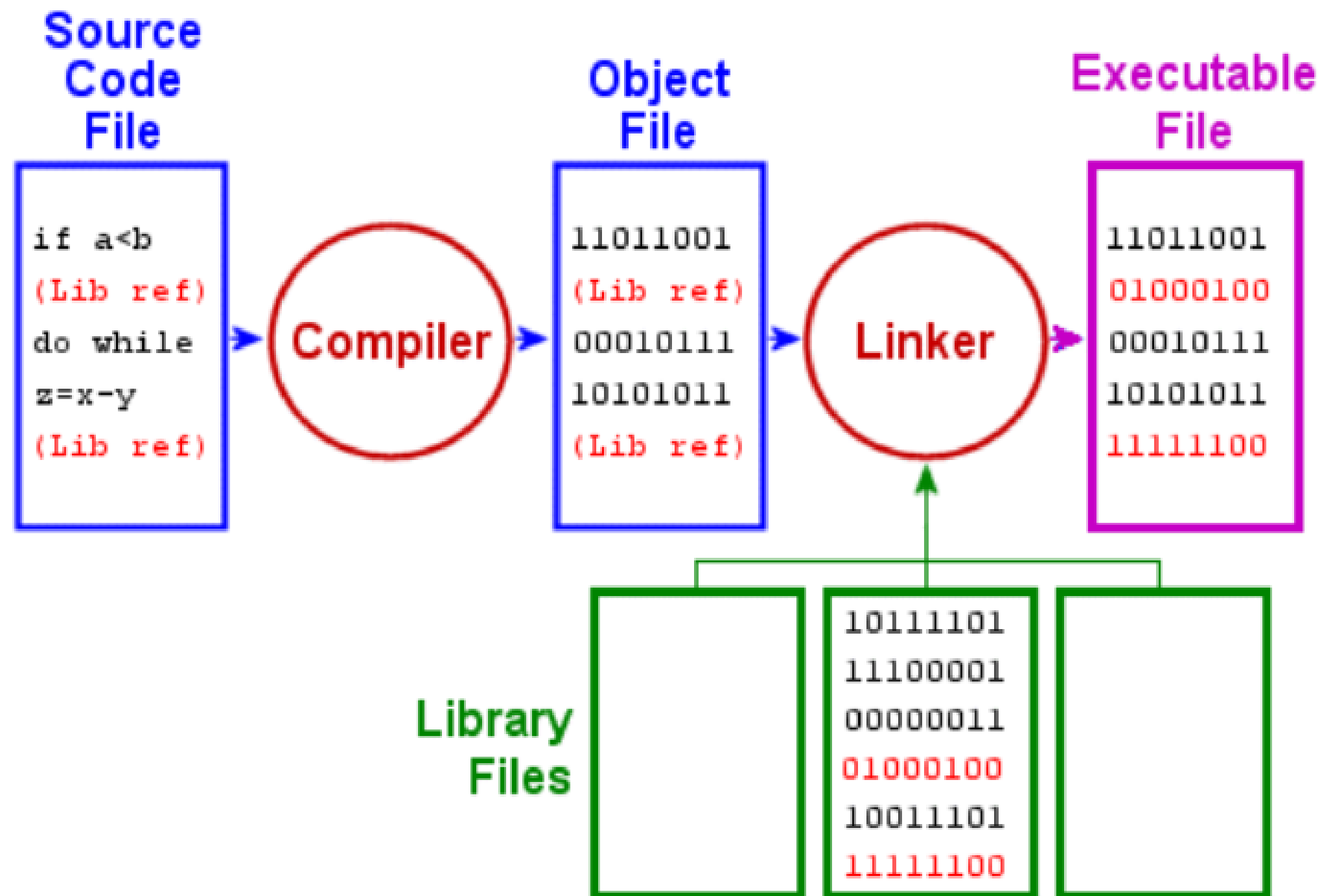
The screenshot shows a VS Code interface for a Codespace named 'practice [Codespaces: bug-free funicular]'. The Explorer panel on the left lists files: about_mc.c, about_me.c, about_me2.c, about_me3.c, main, main.c, main2.c, New Mindmap.mm, practice_stdio.c, and README.md. The main editor shows the content of 'main.c', which is a C program that includes <stdio.h> and prints 'Hello world\n'.

The Terminal panel at the bottom shows a series of git commands and their output, with the commands highlighted in yellow boxes:

```
@prof-kweon →/workspaces/practice (main) $ git add main.c
@prof-kweon →/workspaces/practice (main) $ git commit -m "test"
[main 2bf0989] test
1 file changed, 1 insertion(+), 2 deletions(-)
@prof-kweon →/workspaces/practice (main) $ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 342 bytes | 342.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/prof-kweon/practice
2c7c547..2bf0989 main -> main
@prof-kweon →/workspaces/practice (main) $
```

What is compile?

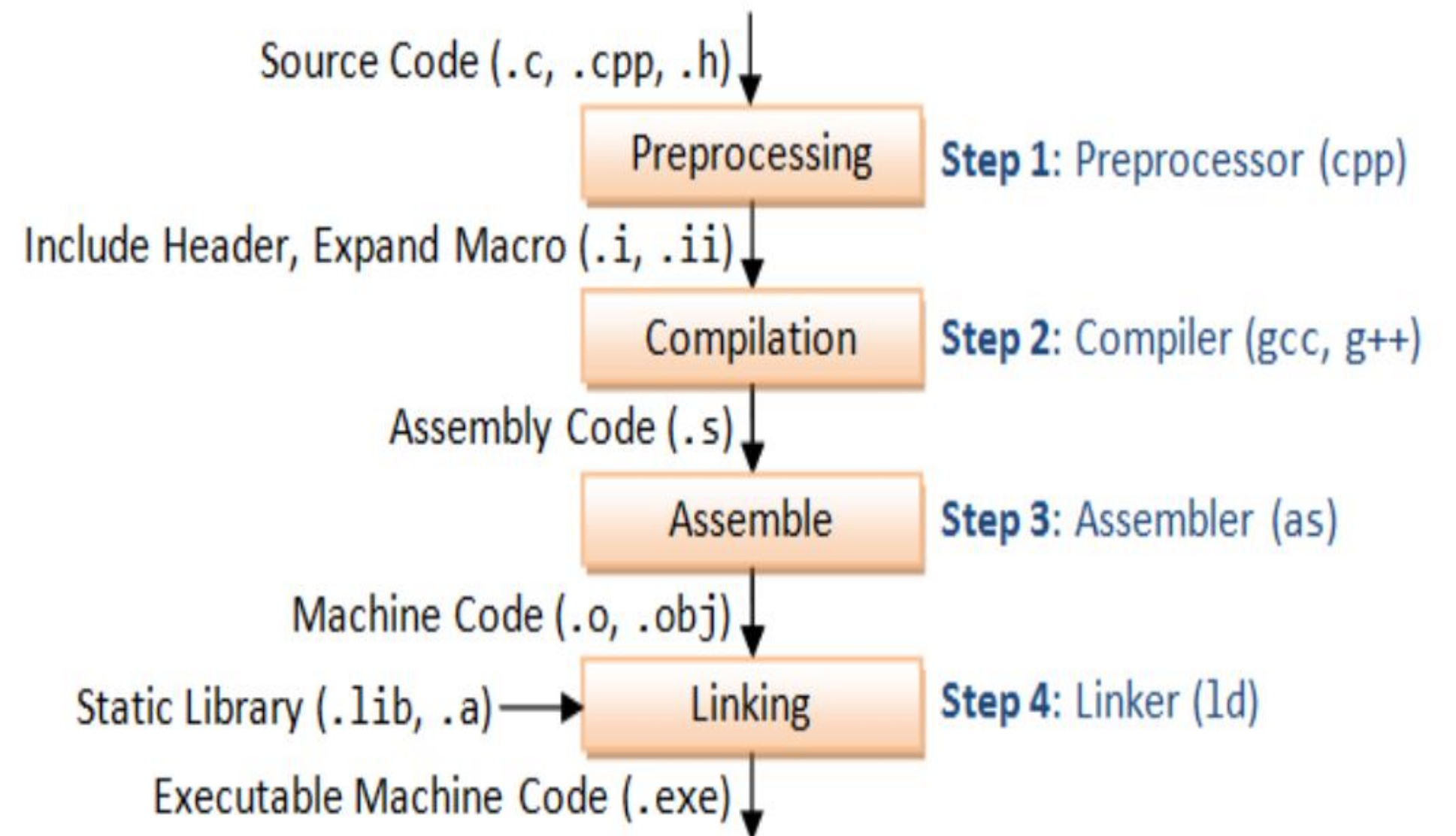
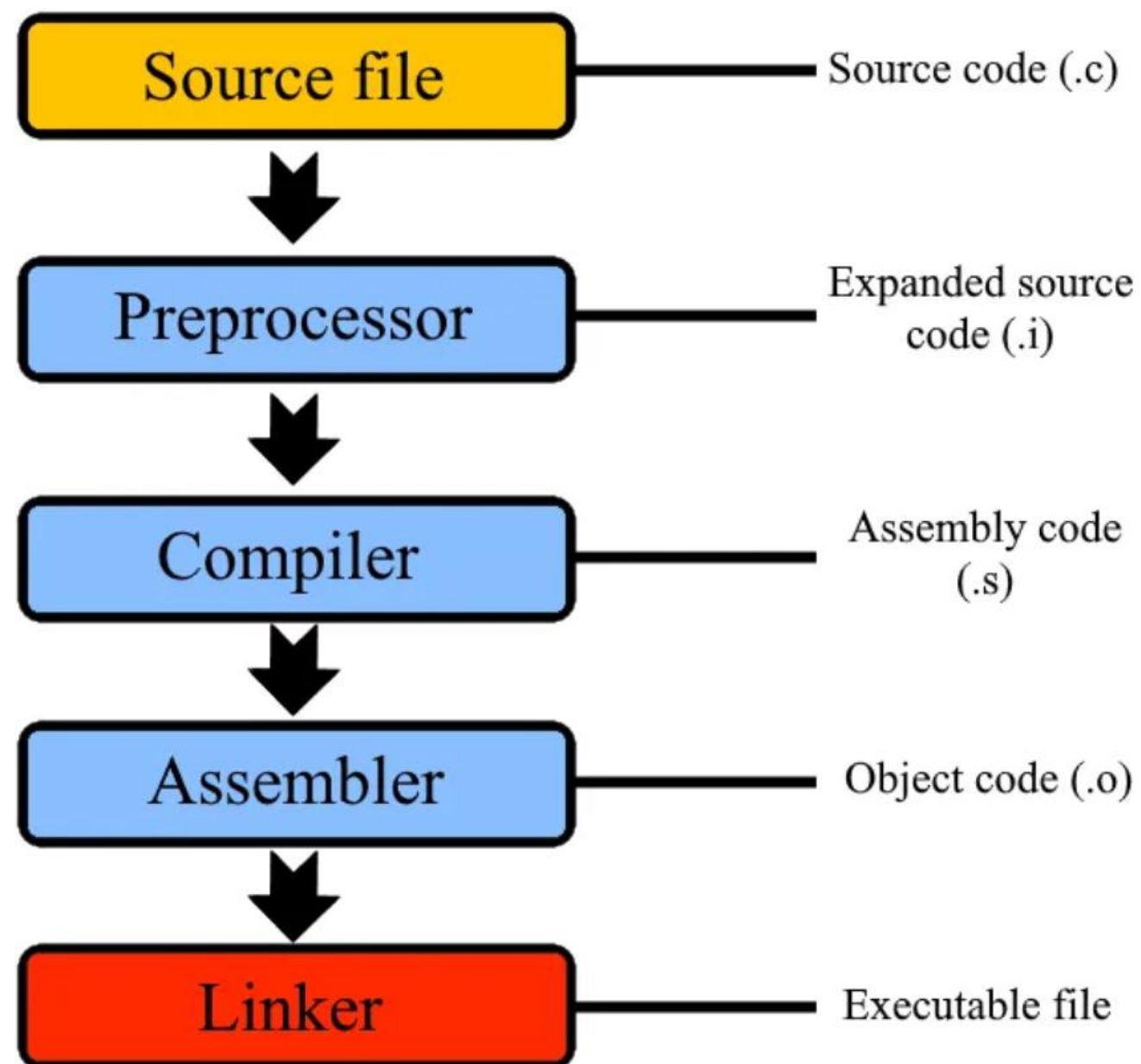
- Translating a high-level language created by humans into a low-level language that a computer can interpret.



Compile/Build Process

- The **Compile Process** is the sequence of steps that a compiler follows to convert **source code** (written in a high-level language like **C, C++, or Java**) into **machine code** (binary instructions that a computer can execute).

Source code : <https://github.com/lattera/glibc/tree/master/stdio-common>



Compile Process

1. main.c : Source code

> Preprocessor : Processes headers (#include) and macros (#define)

2. main.i : Intermediate file containing expanded source code

> Compiler : Edits preprocessed source code into assembly code for a specific processor

`gcc -E main.c -o main.i`

3. main.s : Assembly file

> Assembler : Converts assembly code into machine code

`gcc -S main.i -o main.s`

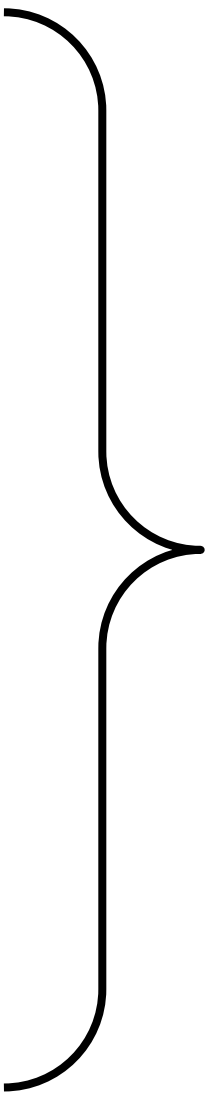
4. main.o : Object file

> Linker : Creates an executable file using the object file and library

`gcc -c main.s -o main.o -> objdump -d main.o / nm main.o`

5. main.exe : Executable file

`gcc main.o -o main -> hexdump -C main.exe`



gcc main.c -o main

Build the first my program

Use the -c flag with gcc to compile the source code into an object file without linking.

gcc -c main.c -o main.o => **main.o**

gcc main.o -o main.exe => **main.exe**

If you don't need an object file and just want an executable, omit the -c flag and use -o flag.

gcc main.c -o main.exe => **main.exe**

Compiler vs. Interpreter

Compiler is a program that translates the entire source code of a programming language into machine code (binary) **before execution**. The resulting executable file can be run independently without requiring the original source code.

Examples of compiled languages:

C, C++, Rust, Go

Characteristics:

- Translates the entire program at once. (Modify-> Compile all again)
- Generates a separate executable file.
- Faster execution since the program is already compiled.
- Errors are detected before execution.

Compiler vs. Interpreter

Interpreter is a program that translates and executes code **line by line** at runtime. It does not generate a separate executable file; instead, it processes the source code dynamically.

Examples of interpreted languages:

Python, JavaScript, Ruby, PHP

Characteristics:

- Translates and runs the code **line by line**.
- No separate executable file; the interpreter is needed each time.
- Slower execution compared to compiled programs.
- Errors are detected **during execution** (runtime).

Compiler vs. Interpreter

Key Differences

Feature	Compiler	Interpreter
Execution Speed	Fast (precompiled)	Slow (line-by-line)
Error Detection	Before execution	During execution
Compile Output	Executable file	No separate executable
Language	C, C++, Rust	Python, JavaScript

Some languages, like **Java**, use both:

- Java source code is compiled into bytecode (.class file).
- The JVM (Java Virtual Machine) interprets the bytecode at runtime.

Compiled from "Hello.java"

```
public class Hello {  
  public Hello();  
  Code:  
    0: aload_0  
    1: invokespecial #1    // Method java/lang/Object."<init>":()V  
    4: return  
  
  public static void main(java.lang.String[]);  
  Code:  
    0: getstatic   #2      // Field  
java/lang/System.out:Ljava/io/PrintStream;  
    3: ldc         #3      // String Hello, world!  
    5: invokevirtual #4     // Method  
java/io/PrintStream.println:(Ljava/lang/String;)V  
    8: return  
}
```

Compiler vs. Interpreter

	Compiler	Interpreter
HTML/CSS	X	X (Browser's direct interpretation)
JavaScript	X	O (JS Engine in browser interpretation)
C/C++	O	X

Binary & ASCII: <https://www.youtube.com/watch?v=Xpk67YzOn5w>

Hex: <https://www.youtube.com/watch?v=5sS7w-CMHkU&t=613s>

Standard input & output

Standard Output (printf)

1. Syntax of `printf()` : Grammar and Structure of language

```
#include <stdio.h>
int printf(const char *format, ...);
```

- The first argument (format) is a string containing text and format specifiers
- The ellipsis (...) represents a variable number of arguments, which are inserted into the format specifier

Specifier	Data Type	Example
<code>%d</code> or <code>%i</code>	Integer (decimal)	<code>printf("%d", 10);</code> → 10
<code>%f</code>	Floating-point (decimal)	<code>printf("%f", 3.14);</code> → 3.140000
<code>%.nf</code>	Floating-point (n decimal places)	<code>printf("%.2f", 3.14159);</code> → 3.14
<code>%c</code>	Single character	<code>printf("%c", 'A');</code> → A
<code>%s</code>	String	<code>printf("%s", "Hello");</code> → Hello
<code>%x</code> or <code>%X</code>	Hexadecimal integer	<code>printf("%x", 255);</code> → ff
<code>%o</code>	Octal integer	<code>printf("%o", 10);</code> → 12
<code>%p</code>	Pointer (memory address)	<code>printf("%p", ptr);</code>
<code>%%</code>	Literal <code>%</code> symbol	<code>printf("%%");</code> → %

Standard Output (printf)

Field width and precision

When printing using printf() , you can specify the size of the field in which data is printed .

output statement	output result	explanation										
printf("%10d", 123);	<table><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td><td>2</td><td>3</td></tr></table>								1	2	3	Width 10, Right aligned
							1	2	3			
printf("%-10d", 123);	<table><tr><td>1</td><td>2</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	2	3								Width 10, Left aligned
1	2	3										

output statement	output result	explanation										
printf("%f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>8</td><td></td><td></td></tr></table>	1	.	2	3	4	5	6	8			6 decimal places
1	.	2	3	4	5	6	8					
printf("%10.3f", 1.23456789);	<table><tr><td></td><td></td><td></td><td></td><td></td><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td></tr></table>						1	.	2	3	5	3 decimal places
					1	.	2	3	5			
printf("%-10.3f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	.	2	3	5						3 decimal places
1	.	2	3	5								
printf("%.3f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	.	2	3	5						3 decimal places
1	.	2	3	5								

```
component_name : TEZ_CLIENT      recovery_enabled : true
component_name : WEBHCAT_SERVER  recovery_enabled : true
component_name : YARN_CLIENT     recovery_enabled : true
component_name : ZKFC            recovery_enabled : true
component_name : ZOOKEEPER_CLIENT recovery_enabled : true
component_name : ZOOKEEPER_SERVER recovery_enabled : true
```



```
component_name : TEZ_CLIENT      recovery_enabled : true
component_name : WEBHCAT_SERVER  recovery_enabled : true
component_name : YARN_CLIENT     recovery_enabled : true
component_name : ZKFC            recovery_enabled : true
component_name : ZOOKEEPER_CLIENT recovery_enabled : true
component_name : ZOOKEEPER_SERVER recovery_enabled : true
```

Standard Output (printf)

3. Format of printf()

- Width and Alignment

You can specify a **minimum width** for the output using numbers.

Default is right-aligned; to left-align, use .

```
printf("%10d\n", 123); // Right-aligned, width 10  
printf("%-10d\n", 123); // Left-aligned, width 10
```

Practice

- Precision for Floating-Point Numbers

```
printf("%.2f\n", 3.14159); // Prints with 2 decimal places
```

Practice

- Padding with Zeros

```
printf("%05d\n", 42); // Pads with zeros up to 5 digits
```

Practice

Standard Output (printf)

4. Using Escape Sequences of printf()

- printf supports escape sequences to control output formatting

Escape Sequence	Meaning	Example Output
<code>\n</code>	Newline	<code>printf("Hello\nWorld");</code> → Hello World
<code>\t</code>	Tab	<code>printf("Hello\tWorld");</code> → Hello World
<code>\\</code>	Backslash	<code>printf("C:\\Program Files\\");</code> → C:\Program Files\
<code>\"</code>	Double Quote	<code>printf("\"Hello\"");</code> → "Hello"

Practice

Standard Output (printf)

control character	name	meaning
<code>\0</code>	null character	
<code>\a</code>	warning(bell)	A "beep" warning sound occurs
<code>\b</code>	backspace	Moves the cursor one character back from its current position.
<code>\t</code>	horizontal tab	Moves the cursor position to the next tab position set on the current line.
<code>\n</code>	newline	Moves the cursor to the beginning of the next line.
<code>\v</code>	vertical tab	Move the cursor to the next set vertical tab position
<code>\f</code>	form feed	It is mainly used in printers to force the printer to turn to the next page.
<code>\r</code>	carriage return	Moves the cursor to the beginning of the current line.
<code>\"</code>	double quotation marks	The original double quotes themselves
<code>\'</code>	single quote	The original single quote itself
<code>\\</code>	backslash	The original backslash itself

Standard Output (printf)

5. Printing Multiple Values

- You can print multiple values in a single printf call by passing multiple

Practice

```
int age = 25;
float pi = 3.14;
printf("Age: %d, Pi: %.2f\n", age, pi);
```

5. Return Value of printf

- printf returns the number of characters printed (excluding \0)

Practice

```
int count = printf("Hello");
printf("\nCharacters printed: %d\n", count);
```

Standard Output (puts)

1. Syntax of **puts()**

```
#include <stdio.h>
int puts(const char *str);
```

- puts prints a string (str) to the console and automatically appends a newline (\n) at the end.
- It is simpler and safer than printf("%s\n", str); because it does not require format specifiers.
- It returns a non-negative integer on success and EOF (-1) on failure.

```
#include <stdio.h>
```

```
int main() {
    puts("Hello, World!");
    return 0;
}
```

Practice

Standard Output (putchar)

1. Syntax of **putchar()**

```
#include <stdio.h>
int putchar(int ch);
```

- putchar prints a single character (ch) to the console.
- It is simpler and safer than printf("%s\n", str); because it does not require format specifiers.
- It returns a non-negative integer on success and EOF (-1) on failure.

```
#include <stdio.h>

int main() {
    putchar('A');
    putchar('\n'); // Manually adding a newline
    return 0;
}
```

Practice

Standard Input

Standard input reads data from an input device, typically the keyboard.
It uses functions like `scanf()`, `getchar()`, and `fgets()` to read user input

[Keyboard typing]



[OS input buffer (stdin)]

↓ (flush when Enter pressed)

[Program function: `scanf/getchar/fgets...`]



[Stored in variable]

User input: hello↵

stdin buffer: 'h' 'e' 'l' 'l' 'o' '\n' '\0'

Standard Input (scanf)

1. Syntax of **scanf()**

```
#include <stdio.h>
int scanf(const char *format, ...);
```

- scanf reads formatted input from stdin (usually the keyboard).
- It requires format specifiers to determine the type of input.
- It stops reading when encountering whitespace (spaces, tabs, newlines, etc.).
- it remains '\n' in buffer. To avoid → getchar();

```
#include <stdio.h>

int main() {
    int age;
    float height;
    printf("Enter your age and height: ");
    scanf("%d %f", &age, &height);
    printf("You are %d years old and %.2f meters tall.\n", age, height);
    return 0;
}
```

Practice

Standard Input (scanf)

2. Key Characteristics of **scanf()**

- Can read multiple values at once.
- Requires the address-of operator (&) for non-string variables.
- Stops reading at the first whitespace character (space, tab, or newline).
- Can cause buffer issues if not used carefully (e.g., failing to handle newline characters properly).

If the user types hello↵:

- The OS puts hello\n in the stdin buffer.
- `scanf("%s", str)` reads hello and stops at the newline.
- The \n stays in the buffer.

Standard Input (scanf)

2. Key Characteristics of **scanf()**

```
int a, b;  
scanf("%d%d", &a, &b); // works the same as "%d %d"
```

Input: 1 2 (multiple spaces or tabs or newline)

- First %d → 1
- Whitespace skipped automatically
- Second %d → 2

Standard Input (getchar)

1. Syntax of **getchar()**

```
#include <stdio.h>
int getchar(void);
```

- getchar reads a single character from stdin.
- It includes whitespace characters like spaces and newlines.
- Returns the character as an unsigned char (cast to int) or EOF on error.

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    ch = getchar();
    printf("You entered: %c\n", ch);
    return 0;
}
```

Practice

Standard Input (Handling Newline (~~W~~n) Issues)

problem: Buffer Retains ~~W~~n

If `getchar` is used after `scanf`, it may read the leftover newline (~~W~~n).

* Find the problem below

```
#include <stdio.h>

int main() {
    int num;
    char ch;

    printf("Enter a number: ");
    scanf("%d", &num); // Reads a number

    printf("Enter a character: ");
    ch = getchar(); // Problem: This reads the newline ('\n') from the buffer!

    printf("Number: %d, Character: %c\n", num, ch);
    return 0;
}
```

Find the problems

Standard Input (fgets)

1. Syntax of **fgets()**

```
#include <stdio.h>
char *fgets(char *str, int n, FILE *stream);
```

- fgets reads a whole line from the input (up to n-1 characters).
- It includes spaces and stops at a newline (\n).
- It prevents buffer overflow by specifying the maximum number of characters.
- It includes '\n' → hello\n\0

```
#include <stdio.h>

int main() {
    char name[50];
    printf("Enter your name: ");
    fgets(name, sizeof(name), stdin);
    printf("Hello, %s", name);
    return 0;
}
```

Practice

Standard Input (fgets)

2. Key Characteristics of **fgets()**

- Reads a full line, including spaces.
- Stops when newline (`\n`) or buffer limit (`n-1` characters) is reached.
- Unlike `scanf`, it does not skip spaces.
- Adds a newline character (`\n`) if the user presses Enter.

Standard Input

Comparison of scanf, getchar, fgets

-

Feature	scanf	getchar	fgets
Reads	Formatted input (integers, floats, strings, etc.)	Single character	Whole line (string)
Stops at	Whitespace (space, tab, newline)	Single character (including spaces)	Newline (<code>\n</code>) or max buffer size
Handles whitespace	Ignores leading spaces	Reads spaces & newlines	Includes spaces, retains newline (<code>\n</code>)
Best for	Numeric input or formatted data	Single character input	Full-line string input
Risk of buffer overflow?	Yes (if not handled properly)	No	No (safe with buffer size)
Newline handling	Left in buffer (needs clearing)	Consumed as input	Stored in string (needs removal if unwanted)

Standard Input

When to use which?

-

Scenario	Best Choice
Reading a single integer or float	<code>scanf</code>
Reading a single character	<code>getchar</code>
Reading an entire line of text (including spaces)	<code>fgets</code>
Reading formatted input (e.g., "Name Age Height")	<code>scanf</code>
Avoiding buffer overflow issues when reading strings	<code>fgets</code>

Standard Input

How each function handles Enter (\n)?

-

Function	Reads \n ?	When does it capture \n ?	How to handle it?
<code>scanf("%d")</code>	✗ No	Skips whitespace, including \n	No need
<code>scanf("%c")</code>	✓ Yes	Captures leftover \n if input before it doesn't consume it	Use <code>" %c"</code> to skip whitespace
<code>getchar()</code>	✓ Yes	Always reads \n if it's in the buffer	Use multiple <code>getchar()</code> calls if needed
<code>fgets()</code>	✓ Yes	Always stores \n in the string (if space allows)	Remove with <code>strcspn()</code>

Problem solving with stdio.h

1. Determine what the input is
 - scanf, fgets, getchar, sscanf

User scenario

- > 1
- > 1 3.14
- > name age email
- > name age email introduction
- > name,age,email,introduction
- > Name:abc, Age:20, Email:abc@email.com, Introduction: I am a boy
- > word sentence number etc

See you next week!

DO NOT miss the classes



Development Environment & setup – Git (Optional)

1. Download Git for Windows : <https://git-scm.com/download/win>
2. Run the Installer : For beginners, the default options are safe (just keep clicking “Next”).
3. Important Setup Options (Recommended)
 - Editor Selection: Choose VS Code
 - PATH Environment: Select “Git from the command line and also from 3rd-party software”.
 - HTTPS Transport: Use the default OpenSSL.
 - Line Ending Conversions: Choose “Checkout Windows-style, commit Unix-style” (recommended default).
4. After installation, you’ll have two main options: Git CMD / PowerShell:
 - Git also works from Windows Command Prompt
5. Verify Installation
`git --version`

Development Environment & setup – git clone (Best way, but difficult)

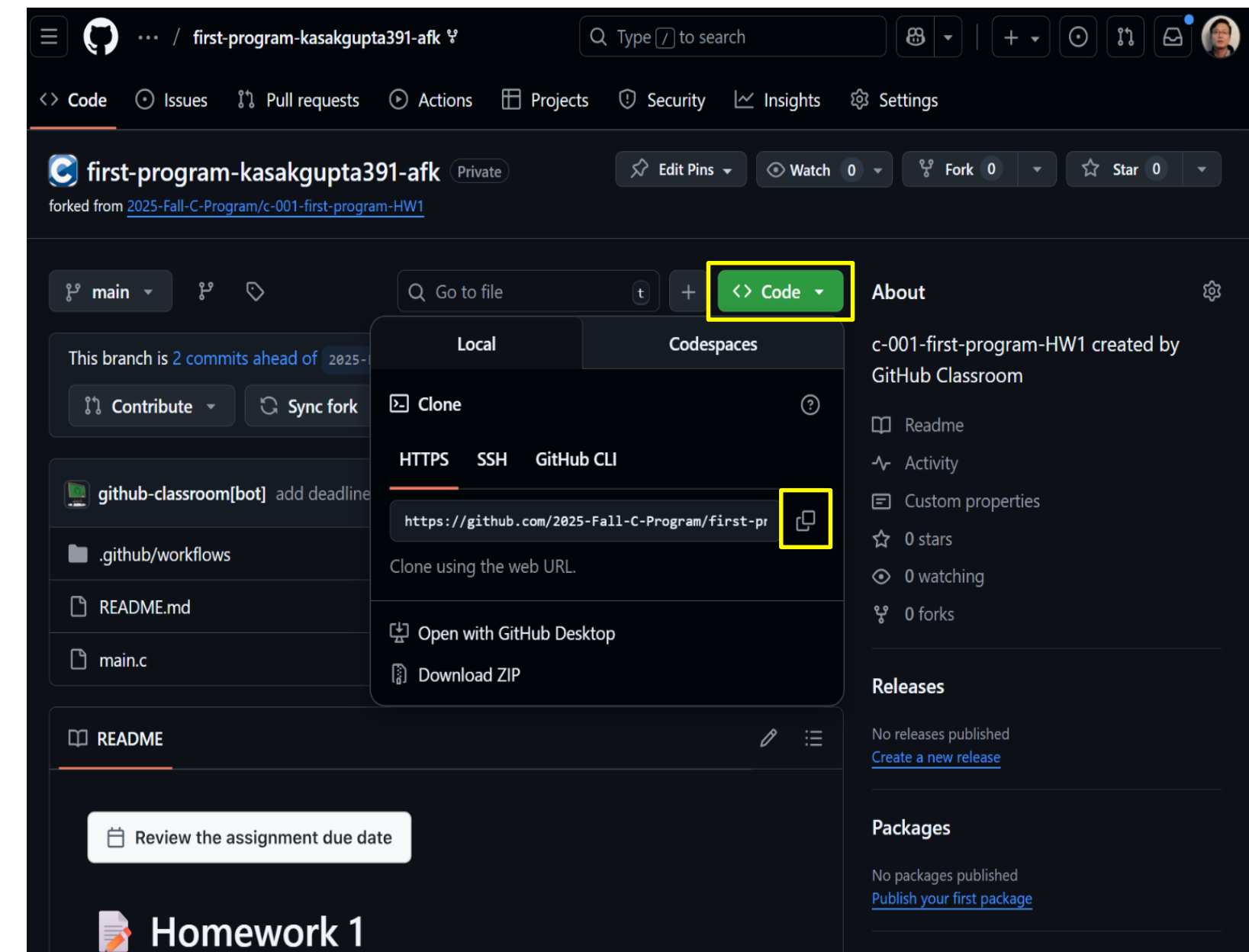
1. Go to your new assignment repository on GitHub.
2. Copy the Repository URL
 - Press “Code”, then copy url
ex) <https://github.com/2025-Fall-C-Program/first-program-kasakgupta391-afk.git>
3. Open **Terminal** (Linux/Mac) or **Git Bash** (Windows).
 - Goto proper folder: `cd c:\c-class`
 - Run: `git clone https://github.com/2025-Fall-C-Program/first-program-kasakgupta391-afk.git`
-> This creates a local folder with the same name as your repo.

```
D:\Projects\wsu>cd c
D:\Projects\wsu\c>dir
Volume in drive D has no label.
Volume Serial Number is E883-9CA4

Directory of D:\Projects\wsu\c

2025-03-05(수) 오후 01:34 <DIR>      .
2025-09-08(월) 오후 01:38 <DIR>      ..
2024-12-23(월) 오후 02:01 <DIR>      test_c
2025-03-05(수) 오후 01:34 <DIR>      the-c-programming-language-second-edition-solutions-master
0 File(s)              0 bytes
4 Dir(s)  1,909,492,338,688 bytes free

D:\Projects\wsu\c>git clone https://github.com/2025-Fall-C-Program/first-program-kasakgupta391-afk.git
Cloning into 'first-program-kasakgupta391-afk'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 14 (delta 0), reused 0 (delta 0), pack-reused 12 (from 2)
Receiving objects: 100% (14/14), 5.65 KiB | 263.00 KiB/s, done.
D:\Projects\wsu\c>
```



Development Environment & setup – git clone (Best way, but difficult)

[Problem]

```
git clone https://github.com/2025-Fall-C-Program/basic-operations-\*\*\*\*\*.git
```

```
Cloning into 'basic-operations-*****' ...
```

```
remote: Repository not found.
```

```
fatal: repository 'https://github.com/2025-Fall-C-Program/basic-operations-\*\*\*\*\*.git/' not found
```

[Solution]

- Authentication

you must be **logged in to GitHub with the account that accepted the Classroom assignment.**

If you use HTTPS cloning, Git will ask for authentication the first time. If you are not logged in properly, you'll see "Repository not found".

- **Assignment not accepted**

In GitHub Classroom, each student must click the assignment link to generate their personal repo.

If you never accepted the assignment, your repo may not exist yet.

Development Environment & setup – git clone (Best way, but difficult)

4. Check new folder after cloning repo.

- dir / ls :
- cd <new_folder> : go to new folder
- cd / pwd :check whole path of new folder)

```
D:\Projects\wsu\c>dir
Volume in drive D has no label.
Volume Serial Number is E883-9CA4

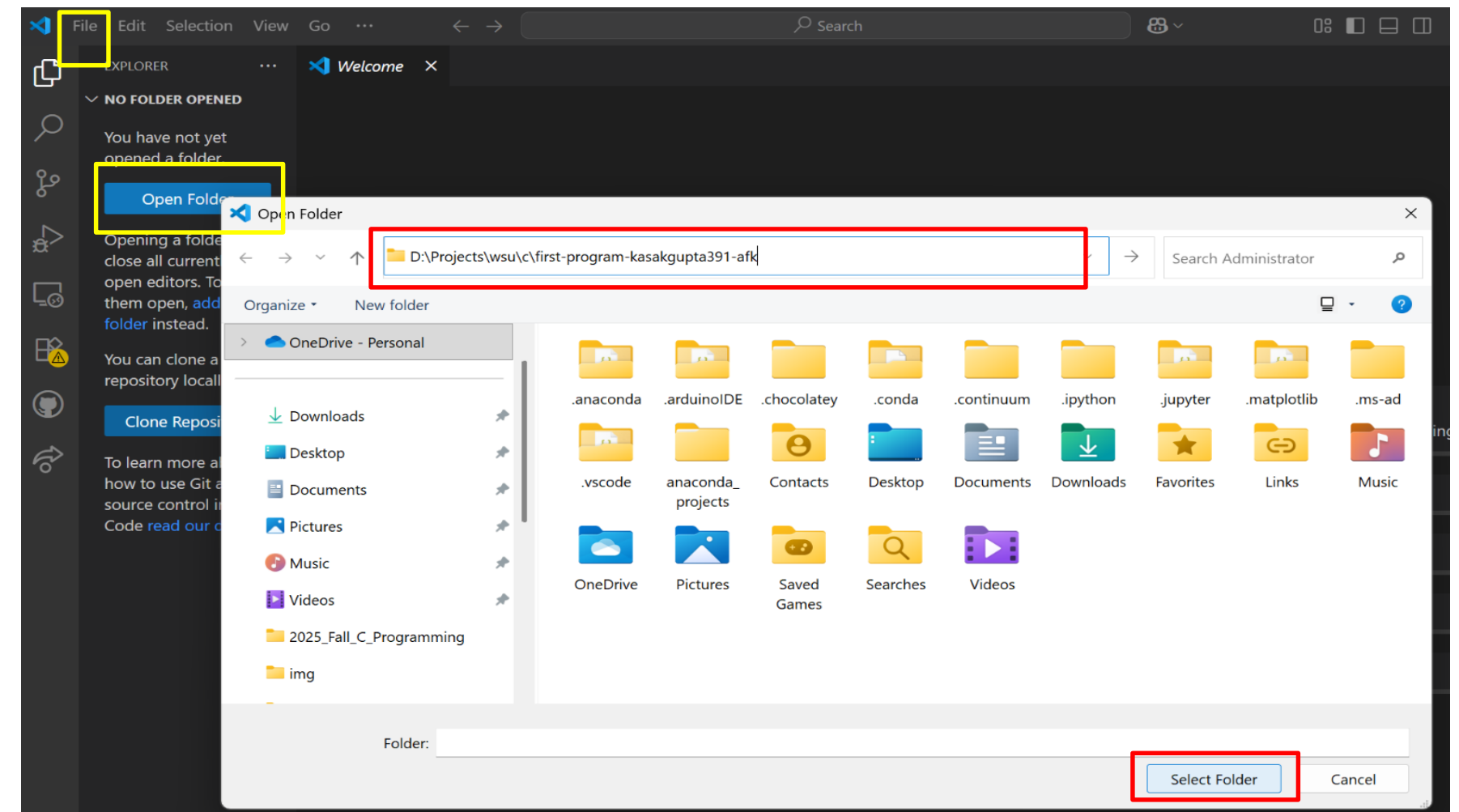
Directory of D:\Projects\wsu\c

2025-09-15(월)   오후 06:00   <DIR>          .
2025-09-08(월)   오후 01:38   <DIR>          .
2025-09-15(월)   오후 06:00   <DIR>          first-program-kasakgupta391-afk
2024-12-23(월)   오후 02:01   <DIR>          test_c
2025-03-05(수)   오후 01:34   <DIR>          the-c-programming-language-second-edition-solutions-master
               0 File(s)                0 bytes
               5 Dir(s)  1,909,491,998,720 bytes free

D:\Projects\wsu\c>
D:\Projects\wsu\c>cd first-program-kasakgupta391-afk
D:\Projects\wsu\c\first-program-kasakgupta391-afk>
D:\Projects\wsu\c\first-program-kasakgupta391-afk>cd
D:\Projects\wsu\c\first-program-kasakgupta391-afk>
D:\Projects\wsu\c\first-program-kasakgupta391-afk>
```

5. Open VSC & open cloned folder (new_folder).

- File > Open folder or Click “Open Folder”
- Find & write the cloned folder > “Select folder”

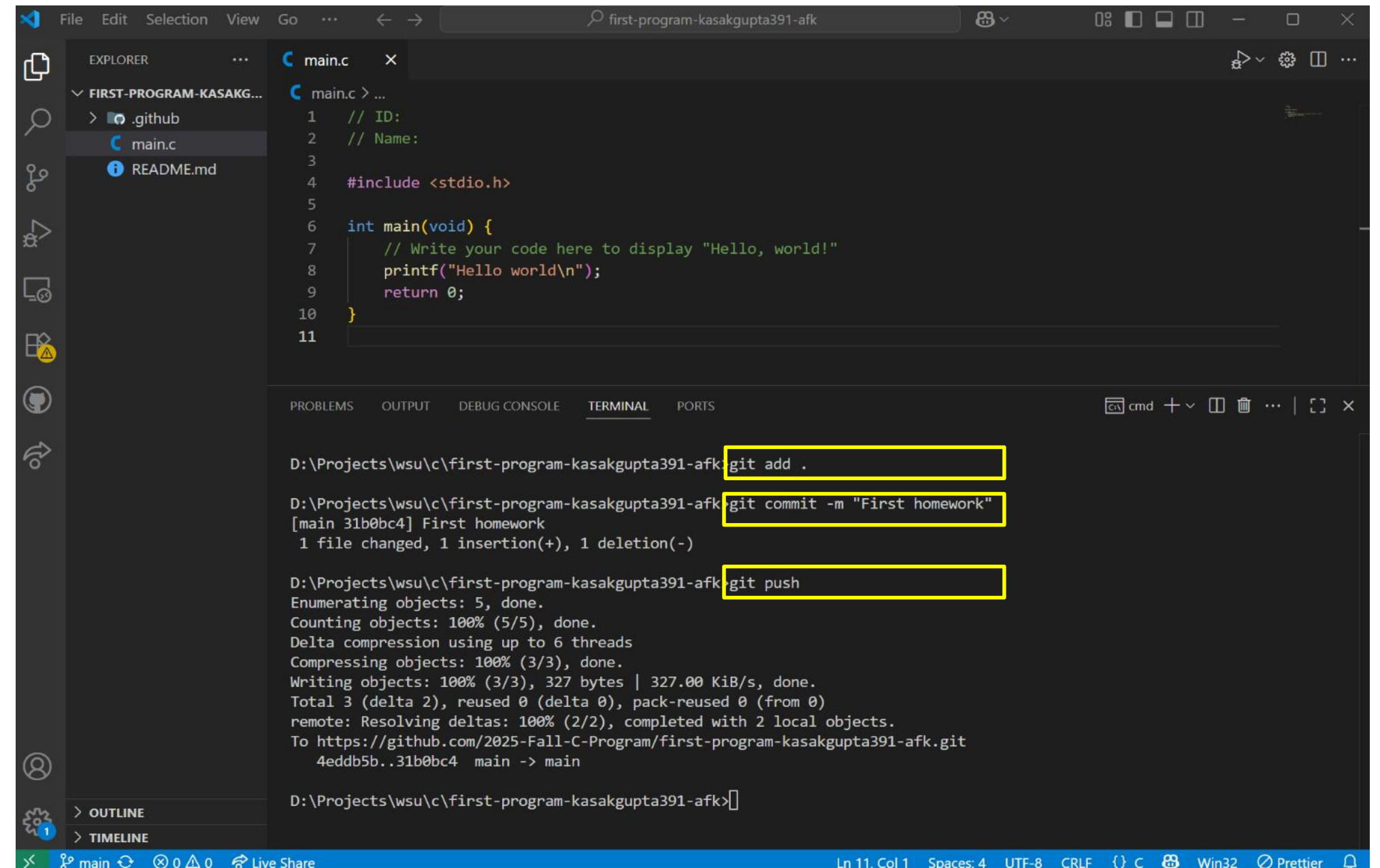
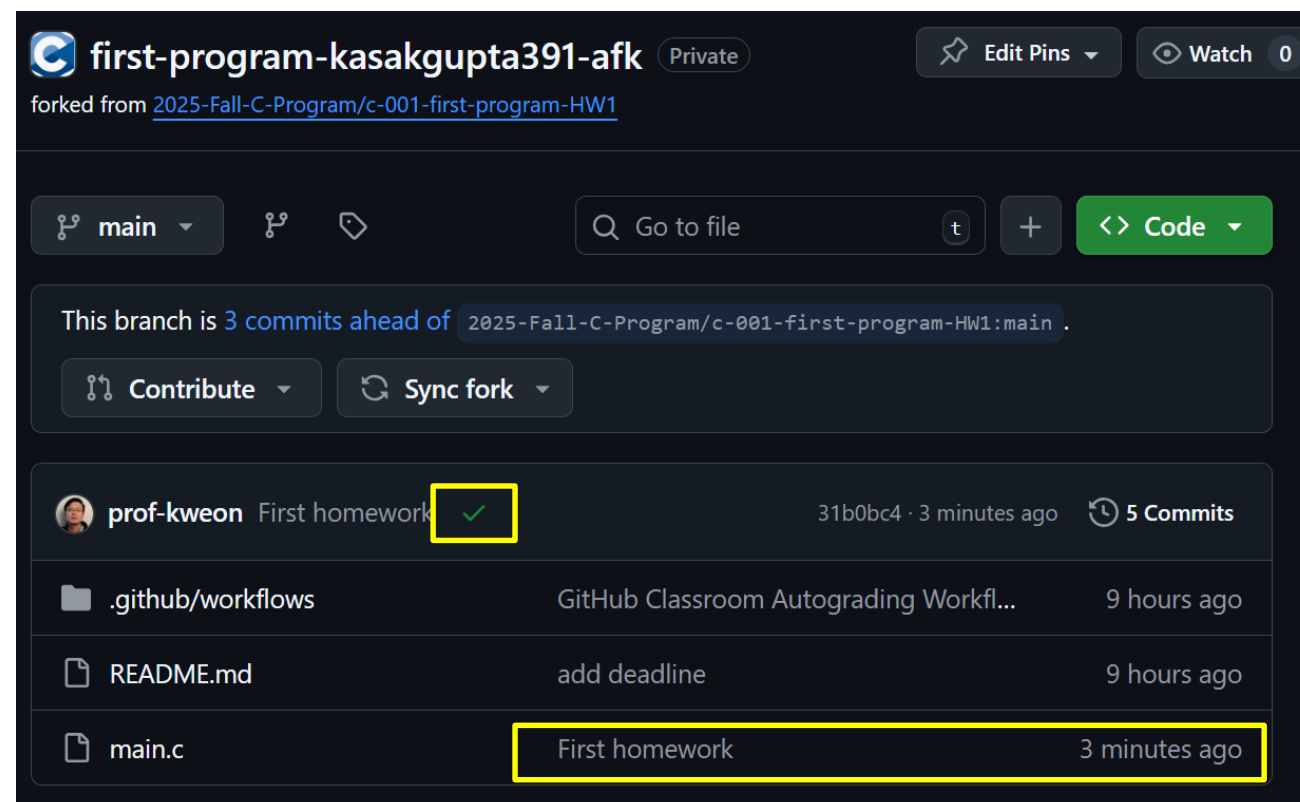


Development Environment & setup – git clone (Best way, but difficult)

6. After modification & verification, type the following.

```
git add . (git add main.c)
git commit -m "msg"
git push
```

7. Verify if your code submitted with message & final result



- If a student modifies their cloned assignment repo, commits, and pushes back to their **private repo**, GitHub Classroom will automatically run the auto-grader and record the score.

Development Environment & setup – git clone (Best way, but difficult)

1. User Identity Setup (Only once per environment)

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

2. Repository Initialization (if starting from scratch. Not using clone) : **Not needed** if you already cloned from GitHub (Classroom assignments)

```
git init
```

3. Check Repository Status

```
git status
```

4. Commit first message

```
git commit -m "Write a clear commit message"
```

5. Connect to Remote Repo (if not already connected)

```
git remote add origin https://github.com/username/repo.git
```

6. Upload the source codes into repo (local commit > Github repo)

```
git push -u origin main
```

Scenario	Commands Needed
First-time setup (new machine or empty Codespace)	git config --global user.name "Name" git config --global user.email "email@example.com" git init (if no repo yet)
Regular workflow (every assignment submission)	git status → git add . → git commit -m "message" → git push

Development Environment & setup – git clone (Best way, but difficult)

Summary workflow

1. Accept assignment link
2. Copy repository URL
3. `git clone <URL>`
4. Edit → Save → Compile & Run locally
5. Update all source codes into assignment repo

`git add .`

`git commit -m "message"`

`git push`

6. Check the result